4

Interpreting Execution Plans

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Objectives

After completing this lesson, you should be able to:

- Gather execution plans
- Display execution plans
- Interpret execution plans

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

What Is an Execution Plan?

- The execution plan of a SQL statement is composed of small building blocks called row sources for serial execution plans.
- The combination of row sources for a statement is called the execution plan.
- By using parent-child relationships, the execution plan can be displayed in a tree-like structure (text or graphical).

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

What Is an Execution Plan?

An execution plan is the output of the optimizer and is presented to the execution engine for implementation. It instructs the execution engine about the operations it must perform for retrieving the data required by a query most efficiently.

The `EXPLAIN PLAN` statement gathers execution plans chosen by the Oracle optimizer for the `SELECT`, `UPDATE`, `INSERT`, and `DELETE` statements. The steps of the execution plan are not performed in the order in which they are numbered. There is a parent-child relationship between steps. The row source tree is the core of the execution plan. It shows the following information:

- An ordering of the tables referenced by the statement
- An access method for each table mentioned in the statement
- A join method for tables affected by join operations in the statement
- Data operations, such as filter, sort, or aggregation

In addition to the row source tree (or data flow tree for parallel operations), the plan table contains information about the following:

- Optimization, such as the cost and cardinality of each operation
- Partitioning, such as the set of accessed partitions
- Parallel execution, such as the distribution method of join inputs

The `EXPLAIN PLAN` results help you determine whether the optimizer selects a particular execution plan, such as nested loops join.

Where to Find Execution Plans?

- `PLAN_TABLE` (SQL Developer or SQL*Plus)
- `V$SQL_PLAN` (Library Cache)
- `V$SQL_PLAN_MONITOR` (11*g*)
- `DBA_HIST_SQL_PLAN` (AWR)
- `STATS$SQL_PLAN` (Statspack)
- SQL management base (SQL plan baselines)
- SQL tuning set
- Trace files generated by `DBMS_MONITOR`
- Event 10053 trace file
- Process state dump trace file since 10*g*R2

ORACLE

4 - 4 Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Where to Find Execution Plans?

There are many ways to retrieve execution plans inside the database. The most well-known ones are listed in the slide:

- The `EXPLAIN PLAN` command enables you to view the execution plan that the optimizer might use to execute a SQL statement. This command is very useful because it outlines the plan that the optimizer may use and inserts it in a table called `PLAN_TABLE` without executing the SQL statement. This command is available from SQL*Plus or SQL Developer.
- `V$SQL_PLAN` provides a way to examine the execution plan for cursors that were recently executed. Information in `V$SQL_PLAN` is very similar to the output of an `EXPLAIN PLAN` statement. However, while `EXPLAIN PLAN` shows a theoretical plan that can be used if this statement was executed, `V$SQL_PLAN` contains the actual plan used.
- `V$SQL_PLAN_MONITOR` displays plan-level monitoring statistics for each SQL statement found in `V$SQL_MONITOR`. Each row in `V$SQL_PLAN_MONITOR` corresponds to an operation of the execution plan that is monitored.
- The Automatic Workload Repository (AWR) infrastructure and Statspack store execution plans of top SQL statements. Plans are recorded into `DBA_HIST_SQL_PLAN` or `STATS$SQL_PLAN`.

- Plan and row source operations are dumped in trace files generated by `DBMS_MONITOR`.
- The SQL management base (SMB) is a part of the data dictionary that resides in the `SYSAUX` tablespace. It stores statement log, plan histories, and SQL plan baselines, as well as SQL profiles.
- The event 10053, which is used to dump cost-based optimizer (CBO) computations may include a plan.
- Starting with Oracle Database 10*g*, Release 2, when you dump process state (or errorstack from a process), execution plans are included in the trace file that is generated.

Viewing Execution Plans

- The `EXPLAIN PLAN` command followed by:
  - `SELECT from PLAN_TABLE`
  - `DBMS_XPLAN.DISPLAY()`
- SQL*Plus Autotrace: `SET AUTOTRACE ON`
- `DBMS_XPLAN.DISPLAY_CURSOR()`
- `DBMS_XPLAN.DISPLAY_AWR()`
- `DBMS_XPLAN.DISPLAY_SQLSET()`
- `DBMS_XPLAN.DISPLAY_SQL_PLAN_BASELINE()`

ORACLE

4 - 6

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Viewing Execution Plans

If you execute the `EXPLAIN PLAN` SQL*Plus command, you can then `SELECT` from the `PLAN_TABLE` to view the execution plan. There are several SQL*Plus scripts available to format the plan table output. The easiest way to view an execution plan is to use the `DBMS_XPLAN` package. The `DBMS_XPLAN` package supplies five table functions:

- `DISPLAY`: To format and display the contents of a plan table
- `DISPLAY_AWR`: To format and display the contents of the execution plan of a stored SQL statement in the AWR
- `DISPLAY_CURSOR`: To format and display the contents of the execution plan of any loaded cursor
- `DISPLAY_SQL_PLAN_BASELINE`: To display one or more execution plans for the SQL statement identified by SQL handle
- `DISPLAY_SQLSET`: To format and display the contents of the execution plan of statements stored in a SQL tuning set

An advantage of using the `DBMS_XPLAN` package table functions is that the output is formatted consistently without regard to the source.

The EXPLAIN PLAN Command

- Generates an optimizer execution plan
- Stores the plan in PLAN_TABLE
- Does not execute the statement itself

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

The EXPLAIN PLAN Command

The EXPLAIN PLAN command is used to generate the execution plan that the optimizer uses to execute a SQL statement. It does not execute the statement, but simply produces the plan that may be used, and inserts this plan into a table. If you examine the plan, you can see how the Oracle Server executes the statement.

Using EXPLAIN PLAN

- First use the EXPLAIN PLAN command to explain a SQL statement.
- Then retrieve the plan steps by querying PLAN_TABLE.

PLAN_TABLE is automatically created as a global temporary table to hold the output of an EXPLAIN PLAN statement for all users. PLAN_TABLE is the default sample output table into which the EXPLAIN PLAN statement inserts rows describing execution plans.

Note: You can create your own PLAN_TABLE using the $ORACLE_HOME/rdbms/admin/utlxplan.sql script if you want to keep the execution plan information for a long term.

The EXPLAIN PLAN Command

```
EXPLAIN PLAN ──────────────────────────────────────────
                          SET STATEMENT_ID
                              = 'text'

──────────────────────────────────────────────────────
        INTO your plan table

──────────────── FOR statement ────────────────────
```

ORACLE

4 - 8

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

The EXPLAIN PLAN Command (continued)

This command inserts a row in the plan table for each step of the execution plan.
In the syntax diagram in the slide, the fields in italics have the following meanings:

Field	Meaning
text	This is an optional identifier for the statement. You should enter a value to identify each statement so that you can later specify the statement that you want explained. This is especially important when you share the plan table with others, or when you keep multiple execution plans in the same plan table.
schema.table	This is the optional name for the output table. The default is PLAN_TABLE.
statement	This is the text of the SQL statement.

The EXPLAIN PLAN Command: Example

```
SQL> EXPLAIN PLAN
  2  SET STATEMENT_ID = 'demo01' FOR
  3  SELECT e.last_name, d.department_name
  4  FROM hr.employees e, hr.departments d
  5  WHERE e.department_id = d.department_id;

Explained.

SQL>
```

Note: The `EXPLAIN PLAN` command does not actually execute the statement.

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

The EXPLAIN PLAN Command: Example

This command inserts the execution plan of the SQL statement in the plan table and adds the optional `demo01` name tag for future reference. You can also use the following syntax:

```
EXPLAIN PLAN
FOR
SELECT e.last_name, d.department_name
   FROM hr.employees e, hr.departments d
WHERE e.department_id =d.department_id;
```

PLAN_TABLE

- PLAN_TABLE:
  - Is automatically created to hold the EXPLAIN PLAN output.
  - You can create your own using utlxplan.sql.
  - Advantage: SQL is not executed
  - Disadvantage: May not be the actual execution plan
- PLAN_TABLE is hierarchical.
- Hierarchy is established with the ID and PARENT_ID columns.

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

PLAN_TABLE

There are various available methods to gather execution plans. Now, you are introduced only to the EXPLAIN PLAN statement. This SQL statement gathers the execution plan of a SQL statement without executing it, and outputs its result in the PLAN_TABLE table. Whatever the method to gather and display the explain plan, the basic format and goal are the same. However, PLAN_TABLE just shows you a plan that might not be the one chosen by the optimizer. PLAN_TABLE is automatically created as a global temporary table and is visible to all users. PLAN_TABLE is the default sample output table into which the EXPLAIN PLAN statement inserts rows describing execution plans. PLAN_TABLE is organized in a tree-like structure and you can retrieve that structure by using both the ID and PARENT_ID columns with a CONNECT BY clause in a SELECT statement. While a PLAN_TABLE table is automatically set up for each user, you can use the utlxplan.sql SQL script to manually create a local PLAN_TABLE in your schema and use it to store the results of EXPLAIN PLAN. The exact name and location of this script depends on your operating system. On UNIX, it is located in the $ORACLE_HOME/rdbms/admin directory. It is recommended that you drop and rebuild your local PLAN_TABLE table after upgrading the version of the database because the columns might change. This can cause scripts to fail or cause TKPROF to fail, if you are specifying the table.

Note: If you want an output table with a different name, first create PLAN_TABLE manually with the utlxplan.sql script, and then rename the table with the RENAME SQL statement.

Displaying from PLAN_TABLE: Typical

```
SQL> EXPLAIN PLAN SET STATEMENT_ID = 'demo01' FOR SELECT * FROM emp
  2  WHERE ename = 'KING';

Explained.

SQL> SET LINESIZE 130
SQL> SET PAGESIZE 0
SQL> select * from table(DBMS_XPLAN.DISPLAY());

Plan hash value: 3956160932
---------------------------------------------------------------------------
| Id  | Operation         | Name | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |      |     1 |    37 |     3   (0)| 00:00:01 |
|*  1 |  TABLE ACCESS FULL| EMP  |     1 |    37 |     3   (0)| 00:00:01 |
---------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter("ENAME"='KING')
```


Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Displaying from PLAN_TABLE: Typical

In the example in the slide, the `EXPLAIN PLAN` command inserts the execution plan of the SQL statement in `PLAN_TABLE` and adds the optional `demo01` name tag for future reference. The `DISPLAY` function of the `DBMS_XPLAN` package can be used to format and display the last statement stored in `PLAN_TABLE`. You can also use the following syntax to retrieve the same result: `SELECT * FROM TABLE(dbms_xplan.display('plan_table','demo01','typical',null));`

The output is the same as shown in the slide. In this example, you can substitute the name of another plan table instead of `PLAN_TABLE` and `demo01` represents the statement `ID`. `TYPICAL` displays the most relevant information in the plan: operation ID, name and option, number of rows, bytes, and optimizer cost. The last parameter for the `DISPLAY` function is the one corresponding to `filter_preds`. This parameter represents a filter predicate or predicates to restrict the set of rows selected from the table where the plan is stored. When value is null (the default), the plan displayed corresponds to the last executed explain plan. This parameter can reference any column of the table where the plan is stored and can contain any SQL construct—for example, subquery or function calls.

Note: Alternatively, you can run the `utlxpls.sql` (or `utlxplp.sql` for parallel queries) script (located in the `ORACLE_HOME/rdbms/admin/` directory) to display the execution plan stored in `PLAN_TABLE` for the last statement explained. This script uses the `DISPLAY` table function from the `DBMS_XPLAN` package.

Displaying from PLAN_TABLE: ALL

```
SQL> select * from table(DBMS_XPLAN.DISPLAY(null,null,'ALL'));

Plan hash value: 3956160932
---------------------------------------------------------------------------
| Id  | Operation         | Name | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |      |     1 |    37 |     3   (0)| 00:00:01 |
|*  1 |  TABLE ACCESS FULL| EMP  |     1 |    37 |     3   (0)| 00:00:01 |
---------------------------------------------------------------------------
Query Block Name / Object Alias (identified by operation id):
-------------------------------------------------------------
   1 - SEL$1 / EMP@SEL$1
Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter("ENAME"='KING')
Column Projection Information (identified by operation id):
-----------------------------------------------------------
 1 - "EMP"."EMPNO"[NUMBER,22], "ENAME"[VARCHAR2,10], "EMP"."JOB"[VARCHAR2,9],
     "EMP"."MGR"[NUMBER,22], "EMP"."HIREDATE"[DATE,7], "EMP"."SAL"[NUMBER,22],
     "EMP"."COMM"[NUMBER,22], "EMP"."DEPTNO"[NUMBER,22]
```

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Displaying from PLAN_TABLE: ALL

Here you use the same EXPLAIN PLAN command example as in the previous slide. The ALL option used with the DISPLAY function allows you to output the maximum user level information. It includes information displayed with the TYPICAL level, with additional information such as PROJECTION, ALIAS, and information about REMOTE SQL, if the operation is distributed.

For finer control on the display output, the following keywords can be added to the format parameter to customize its default behavior. Each keyword either represents a logical group of plan table columns (such as PARTITION) or logical additions to the base plan table output (such as PREDICATE). Format keywords must be separated by either a comma or a space:

- ROWS: If relevant, shows the number of rows estimated by the optimizer
- BYTES: If relevant, shows the number of bytes estimated by the optimizer
- COST: If relevant, shows optimizer cost information
- PARTITION: If relevant, shows partition pruning information
- PARALLEL: If relevant, shows PX information (distribution method and table queue information)
- PREDICATE: If relevant, shows the predicate section
- PROJECTION: If relevant, shows the projection section

The EXPLAIN PLAN Command

```
EXPLAIN PLAN
                    SET STATEMENT_ID
                        = 'text'

INTO your plan table

FOR statement
```


Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Displaying from PLAN_TABLE: ALL (continued)

- ALIAS: If relevant, shows the "Query Block Name/Object Alias" section
- REMOTE: If relevant, shows the information for the distributed query (for example, remote from serial distribution and remote SQL)
- NOTE: If relevant, shows the note section of the explain plan

If the target plan table also stores plan statistics columns (for example, it is a table used to capture the content of the fixed view V$SQL_PLAN_STATISTICS_ALL), additional format keywords can be used to specify which class of statistics to display when using the DISPLAY function. These additional format keywords are IOSTATS, MEMSTATS, ALLSTATS and LAST.

Note: Format keywords can be prefixed with the "–" sign to exclude the specified information. For example, "–PROJECTION" excludes projection information.

Displaying from PLAN_TABLE: ADVANCED

```
select plan_table_output from table(DBMS_XPLAN.DISPLAY(null,null,'ADVANCED
 -PROJECTION -PREDICATE -ALIAS'));
Plan hash value: 3956160932
---------------------------------------------------------------------------
| Id  | Operation         | Name | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |      |     1 |    37 |     3   (0)| 00:00:01 |
|   1 |  TABLE ACCESS FULL| EMP  |     1 |    37 |     3   (0)| 00:00:01 |
---------------------------------------------------------------------------
Outline Data
-------------
  /*+
      BEGIN_OUTLINE_DATA
      FULL(@"SEL$1" "EMP"@"SEL$1")
      OUTLINE_LEAF(@"SEL$1")
      ALL_ROWS
      DB_VERSION('11.1.0.6')
      OPTIMIZER_FEATURES_ENABLE('11.1.0.6')
      IGNORE_OPTIM_EMBEDDED_HINTS
      END_OUTLINE_DATA
  */
```


Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Displaying from PLAN_TABLE: ADVANCED

The ADVANCED format is available only from Oracle Database 10*g,* Release 2 and later versions.

This output format includes all sections from the ALL format plus the outline data that represents a set of hints to reproduce that particular plan.

This section may be useful if you want to reproduce a particular execution plan in a different environment.

This is the same section, which is displayed in the trace file for event 10053.

Note: When the ADVANCED format is used with V$SQL_PLAN, there is one more section called Peeked Binds (identified by position).

Explain Plan Using SQL Developer

hr | scott | sys_connection

0.007 seconds

```
select e.email, d.department_name
FROM EMPLOYEES e, departments d
where email like 'A%';
```

Script Output | Explain Plan

0.007 seconds

OPERATION	OBJECT_NAME	OPTIONS	COST
SELECT STATEMENT			8
MERGE JOIN		CARTESIAN	8
INDEX	EMP_EMAIL_UK	RANGE SCAN	1
Access Predicates			
EMAIL LIKE 'A%'			
Filter Predicates			
EMAIL LIKE 'A%'			
BUFFER		SORT	7
TABLE ACCESS	DEPARTMENTS	FULL	2

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Explain Plan Using SQL Developer

The Explain Plan icon generates the execution plan, which you can see in the Explain tab. An execution plan shows a row source tree with the hierarchy of operations that make up the statement. For each operation, it shows the ordering of the tables referenced by the statement, access method for each table mentioned in the statement, join method for tables affected by join operations in the statement, and data operations such as filter, sort, or aggregation. In addition to the row source tree, the plan table displays information about optimization (such as the cost and cardinality of each operation), partitioning (such as the set of accessed partitions), and parallel execution (such as the distribution method of join inputs).

AUTOTRACE

- Is a SQL*Plus and SQL Developer facility
- Was introduced with Oracle 7.3
- Needs a PLAN_TABLE
- Needs the PLUSTRACE role to retrieve statistics from some V$ views
- By default, produces the execution plan and statistics after running the query
- May not be the execution plan used by the optimizer when using bind peeking (recursive EXPLAIN PLAN)

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

AUTOTRACE

When running SQL statements under SQL*Plus or SQL Developer, you can automatically get a report on the execution plan and the statement execution statistics. The report is generated after successful SQL DML (that is, SELECT, DELETE, UPDATE, and INSERT) statements. It is useful for monitoring and tuning the performance of these statements.

To use this feature, you must have a PLAN_TABLE available in your schema, and then have the PLUSTRACE role granted to you. The database administrator (DBA) privileges are required to grant the PLUSTRACE role. The PLUSTRACE role is created and granted to the DBA role by running the supplied $ORACLE_HOME/sqlplus/admin/plustrce.sql script.

On some versions and platforms, this is run by the database creation scripts. If this is not the case on your platform, connect as SYSDBA and run the plustrce.sql script.

The PLUSTRACE role contains the select privilege on three V$ views. These privileges are necessary to generate AUTOTRACE statistics.

AUTOTRACE is an excellent diagnostic tool for SQL statement tuning. Because it is purely declarative, it is easier to use than EXPLAIN PLAN.

Note: The system does not support EXPLAIN PLAN for statements performing implicit type conversion of date bind variables. With bind variables in general, the EXPLAIN PLAN output might not represent the real execution plan.

The AUTOTRACE Syntax

```
                                  OFF
SET AUTOTRACE ------------------- ON ------------------
                                  TRACE[ONLY]
-------------------------------------------------------
                                  EXPLAIN
                                  STATISTICS

SHOW AUTOTRACE -------------------------------------
```


Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

The AUTOTRACE Syntax

You can enable AUTOTRACE in various ways using the syntax shown in the slide. The command options are as follows:

- OFF: Disables autotracing SQL statements
- ON: Enables autotracing SQL statements
- TRACE or TRACE[ONLY]: Enables autotracing SQL statements and suppresses statement output
- EXPLAIN: Displays execution plans, but does not display statistics
- STATISTICS: Displays statistics, but does not display execution plans

Note: If both the EXPLAIN and STATISTICS command options are omitted, execution plans and statistics are displayed by default.

AUTOTRACE: Examples

- To start tracing statements using AUTOTRACE:

```
SQL> set autotrace on
```

- To display the execution plan only without execution:

```
SQL> set autotrace traceonly explain
```

- To display rows and statistics:

```
SQL> set autotrace on statistics
```

- To get the plan and the statistics only (suppress rows):

```
SQL> set autotrace traceonly
```

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

AUTOTRACE: Examples

You can control the report by setting the AUTOTRACE system variable. The following are some examples:

- SET AUTOTRACE ON: The AUTOTRACE report includes both the optimizer execution plan and the SQL statement execution statistics.
- SET AUTOTRACE TRACEONLY EXPLAIN: The AUTOTRACE report shows only the optimizer execution path without executing the statement.
- SET AUTOTRACE ON STATISTICS: The AUTOTRACE report shows the SQL statement execution statistics and rows.
- SET AUTOTRACE TRACEONLY: This is similar to SET AUTOTRACE ON, but it suppresses the printing of the user's query output, if any. If STATISTICS is enabled, the query data is still fetched, but not printed.
- SET AUTOTRACE OFF: No AUTOTRACE report is generated. This is the default.

AUTOTRACE: Statistics

```
SQL> show autotrace
autotrace OFF
SQL> set autotrace traceonly statistics
SQL> SELECT * FROM oe.products;

288 rows selected.

Statistics
----------------------------------------------------------
       1334  recursive calls
          0  db block gets
        686  consistent gets
        394  physical reads
          0  redo size
     103919  bytes sent via SQL*Net to client
        629  bytes received via SQL*Net from client
         21  SQL*Net roundtrips to/from client
         22  sorts (memory)
          0  sorts (disk)
        288  rows processed
```

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

AUTOTRACE: Statistics

The statistics are recorded by the server when your statement executes and indicate the system resources required to execute your statement. The results include the following statistics:

- `recursive calls` is the number of recursive calls generated at both the user and system level. Oracle Database maintains tables used for internal processing. When Oracle Database needs to make a change to these tables, it internally generates an internal SQL statement, which in turn generates a recursive call.
- `db block gets` is the number of times a `CURRENT` block was requested.
- `consistent gets` is the number of times a consistent read was requested for a block.
- `physical reads` is the total number of data blocks read from disk. This number equals the value of “physical reads direct” plus all reads into buffer cache.
- `redo size` is the total amount of redo generated in bytes.
- `bytes sent via SQL*Net to client` is the total number of bytes sent to the client from the foreground processes.
- `bytes received via SQL*Net from client` is the total number of bytes received from the client over Oracle Net.

- `SQL*Net roundtrips to/from client` is the total number of Oracle Net messages sent to and received from the client.

Note: The statistics printed by `AUTOTRACE` are retrieved from `V$SESSTAT`.

- `sorts (memory)` is the number of sort operations that were performed completely in memory and did not require any disk writes.
- `sorts (disk)` is the number of sort operations that required at least one disk write.
- `rows processed` is the number of rows processed during the operation.

The client referred to in the statistics is SQL*Plus. Oracle Net refers to the generic process communication between SQL*Plus and the server, regardless of whether Oracle Net is installed. You cannot change the default format of the statistics report.

Note: `db block gets` indicates reads of the current block from the database. `consistent gets` are reads of blocks that must satisfy a particular system change number (SCN). `physical reads` indicates reads of blocks from disk. `db block gets` and `consistent gets` are the two statistics that are usually monitored. These should be low compared to the number of rows retrieved. Sorts should be performed in memory rather than on disk.